

Homework 3: root finding and optimization

Submission

Due: Oct 12, 2026

Submit the following three files in your directory:

1. `lastname_firstname_hw03.ipynb`: one executed Jupyter notebook containing all code and markdown text;
2. `lastname_firstname_hw03.pdf`: a PDF compiled or exported from the executed notebook; and
3. `requirements.txt`: a short list of the packages needed to run the notebook, with version numbers.

Your notebook must run from top to bottom in a fresh session without manual edits. Use the supplied random seeds. Report requested numerical results with at least three significant digits. Keep written explanations concise, but explain the numerical behavior rather than reporting outputs alone.

Question 1: Solving a likelihood equation (30 points)

Consider a logistic model with a known offset z_i and an unknown intercept θ :

$$\Pr(Y_i = 1) = \frac{1}{1 + \exp\{-(\theta + z_i)\}}.$$

The derivative of the negative log-likelihood is

$$g(\theta) = \sum_{i=1}^n \{p_i(\theta) - y_i\}, \quad g'(\theta) = \sum_{i=1}^n p_i(\theta) \{1 - p_i(\theta)\}.$$

Generate the data and define the functions using:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import expit

rng = np.random.default_rng(123)
n = 2000
offset = rng.normal(scale=1.5, size=n)
theta_true = -0.7
y = rng.binomial(1, expit(theta_true + offset))

def g(theta):
    return np.sum(expit(theta + offset) - y)

def g_prime(theta):
    p = expit(theta + offset)
    return np.sum(p * (1 - p))
```

1. Evaluate $g(\theta)$ on 200 equally spaced points from -5 to 5 . Plot the curve with a horizontal line at zero. Report $g(-5)$ and $g(5)$. Explain why these results suggest that a unique root lies in $[-5, 5]$.

- Implement the bisection method on $[-5, 5]$. Stop when either $|g(\theta_k)| < 10^{-8}$ or the bracket width is below 10^{-8} . Report the estimated root, final residual, number of iterations, and final bracket width. Do not call a prewritten root-finding function for this part.
- Implement Newton's method for at most 20 iterations. Stop when $|g(\theta_k)| < 10^{-8}$, or declare failure if the derivative is below 10^{-12} or an iterate becomes nonfinite. Run it from $\theta_0 = 0$ and $\theta_0 = 3$. Report the iterate sequence for each initialization and explain why Newton succeeds from one starting point but fails from the other. Compare this behavior with bisection.

Question 2: Step size and curvature in optimization (35 points)

Consider the positive-definite quadratic

$$f(x) = \frac{1}{2}x^\top Ax - b^\top x, \quad \nabla f(x) = Ax - b.$$

Use the following setup:

```
import numpy as np

angle = np.pi / 6
R = np.array([
    [np.cos(angle), -np.sin(angle)],
    [np.sin(angle),  np.cos(angle)],
])
A = R @ np.diag([1.0, 100.0]) @ R.T
b = np.array([1.0, -1.0])
x0 = np.array([3.0, -2.0])

def objective(x):
    return 0.5 * x @ A @ x - b @ x

def gradient(x):
    return A @ x - b
```

- Compute the eigenvalues and condition number of A , and obtain the exact minimizer x^* by solving $Ax^* = b$. Use $\lambda_{\max}(A)$ to state the interval of fixed step sizes for which gradient descent converges on this quadratic.
- Starting from x_0 , run 100 gradient-descent iterations using each step size

$$\alpha \in \{0.005, 0.015, 0.021\}.$$

Plot $f(x_k) - f(x^*)$ against iteration on a logarithmic vertical scale. Report the final objective gap and gradient norm for each step size. Explain the three trajectories using the convergence interval from Part 1.

- Implement gradient descent with an Armijo backtracking line search using initial step size 1, contraction factor 0.5, and Armijo constant 10^{-4} . Stop when $\|\nabla f(x_k)\|_2 < 10^{-6}$ or after 2,000 iterations. Report the final point, objective gap, gradient norm, iteration count, and number of objective evaluations.

Question 3: Constrained and stochastic optimization of mixture weights (35 points)

For fixed component densities $q_j(y_i)$, estimate mixture weights by minimizing

$$f(w) = -\frac{1}{n} \sum_{i=1}^n \log \left\{ \sum_{j=1}^K w_j q_j(y_i) \right\}, \quad w_j \geq 0, \quad \sum_{j=1}^K w_j = 1.$$

Use the following data and helper functions:

```

import numpy as np

rng = np.random.default_rng(789)
n, K = 3000, 5
means = np.linspace(-3, 3, K)
true_weights = np.array([0.05, 0.15, 0.45, 0.25, 0.10])
component = rng.choice(K, size=n, p=true_weights)
y = rng.normal(means[component], 1.0)
Q = np.exp(-0.5 * (y[:, None] - means)**2) / np.sqrt(2 * np.pi)

def objective(w, q=Q):
    return -np.mean(np.log(np.maximum(q @ w, 1e-300)))

def gradient(w, q=Q):
    density = np.maximum(q @ w, 1e-300)
    return -(q.T @ (1 / density)) / len(density)

def project_simplex(v):
    u = np.sort(v)[::-1]
    shifted_sum = np.cumsum(u) - 1
    rho = np.flatnonzero(u - shifted_sum / np.arange(1, len(v) + 1) > 0)[-1]
    return np.maximum(v - shifted_sum[rho] / (rho + 1), 0)

```

1. Starting from equal weights, solve the problem using `scipy.optimize.minimize` with `method="SLSQP"`, bounds $0 \leq w_j \leq 1$, the equality constraint $\sum_j w_j = 1$, and the supplied gradient. Report the weights, objective, iterations, success message, sum-to-one error, and smallest weight.
2. Define a numerically stable softmax transformation

$$w_j(\mathbf{z}) = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

and minimize `objective(softmax(z))` with BFGS, beginning at $\mathbf{z} = \mathbf{0}$. Report the weights, objective, iterations, and sum-to-one error, and smallest weight. Compare the result with SLSQP and briefly discuss.

3. Implement projected gradient descent from equal weights using a fixed step size of 0.05 for 1,000 iterations. Plot the objective every ten iterations. Report the final weights, objective, sum-to-one error, smallest weight, and relative difference from the SLSQP weights.

Grading guide

Criterion	Points
Correct implementation and numerical results	50
Sound analysis and interpretation	25
Clear figures, tables, and concise communication	15
Reproducibility and complete submission	10
Total	100

Minor differences caused by software versions and timing variability are acceptable. Numerical results without written interpretation will receive limited credit.